S.I.G.S.M

Ingeniería de Software

Empre S.A.S.

Rol	Apellido	Nombre	C.I	Email
Coordinador	Cabrera	Martin	4919741-2	empre.sa.utu.isbo@gmail.com
Subcoordinador	Sena	Angel	5341149-0	jeansangel2003@gmail.com

Docente: Cairús, Brandon

Fecha de culminación

09/11/2026

TERCERA ENTREGA

1. Revisión del Diagrama de Clases..... 5

1.1 Introducción..... 5

1.2 Cambios respecto a la segunda entrega..... 5

1.3 Identificación de las clases implementadas..... 6

1.3.1 Configuración y acceso a datos..... 7

1.3.2 Módulo de gestión de traslados..... 7

1.3.3 Módulo de encuestas..... 8

1.3.4 Administración de funcionarios..... 8

1.3.5 Organización de las clases..... 8

1.4 Relaciones y dependencias entre clases..... 9

1.4.1 Relación con la clase Database..... 9

1.4.2 Dependencias de los servicios..... 10

1.4.3 Relación con los handlers y endpoints..... 10

1.4.4 Representación de las relaciones en el diagrama..... 11

1.5 Diagrama de clases actualizado..... 11

1.6 Justificación de la estructura modular..... 14

2. Plan de Testing..... 15

2.1 Introducción y objetivos..... 15

2.2 Alcance y metodología de pruebas..... 17

2.3 Entorno y herramientas utilizadas..... 18

2.4 Pruebas funcionales..... 19

2.4.1 Autenticación y gestión de funcionarios..... 20

2.4.2 Gestión documental..... 20

2.4.3 Gestión de encuestas..... 21

2.4.4 Gestión y seguimiento de traslados..... 21

2.5 Tablas de decisión..... 22
2.6 Casos de prueba basados en casos de uso..... 24
2.7 Registro de resultados e incidencias..... 25
2.8 Conclusiones del testing..... 26
3. Comparación de Métricas de Tamaño y Función..... 27
3.1 Introducción y objetivo de la comparación..... 27
3.2 Métricas obtenidas en la segunda entrega..... 28
3.3 Actualización de las métricas de tamaño..... 29
3.4 Actualización de las métricas de función..... 31
3.4.1 Puntos de función sin ajustar (PFSA)..... 32
3.4.2 Factor de ajuste (FA)..... 34
3.4.3 Puntos de función ajustados (PFA)..... 36
3.5 Comparación de resultados entre entregas..... 37
3.6 Comparación de la estimación del esfuerzo..... 38
3.7 Análisis de las variaciones..... 41
3.7 Conclusiones..... 43
4. Análisis FODA ponderado..... 44
4.1 Introducción..... 44
4.2 Criterios de ponderación..... 44
4.3 Matriz de Evaluación de Factores Internos (EFI)..... 45
4.4 Matriz de Evaluación de Factores Externos (EFE)..... 47
4.5 Interpretación de resultados..... 49
4.6 Estrategias derivadas del análisis..... 50
4.7 Conclusiones del análisis FODA ponderado..... 50
5. Retrospectiva final del proyecto..... 51

5.1 Introducción y objetivos..... 51
5.2 Retrospectiva grupal..... 52
5.2.1 Organización y distribución de tareas..... 52
5.2.2 Dificultades encontradas y soluciones aplicadas..... 53
5.2.3 Aspectos positivos y aprendizajes del equipo..... 55
5.2.4 Aspectos a mejorar..... 56
5.3 Retrospectivas individuales..... 57
5.3.1 Martín Cabrera..... 57
5.3.2 Ángel Sena..... 58
5.4 Conclusiones de la retrospectiva..... 58
6. Evaluación de la experiencia del cliente..... 59
6.1 Introducción y objetivos..... 59
6.2 Presentación y demostración del sistema..... 60
6.3 Valoración y comentarios recibidos..... 61
6.4 Sugerencias de mejora..... 62
6.5 Análisis de la retroalimentación..... 63
6.6 Conclusiones de la evaluación..... 65
7. Conclusiones generales del proyecto..... 66
7.1 Cumplimiento de los objetivos..... 66
7.2 Resultados y funcionalidades desarrolladas..... 67
7.3 Limitaciones y funcionalidades pendientes..... 68
7.4 Posibles mejoras y ampliaciones futuras..... 69
7.5 Reflexión y cierre del proyecto..... 70

1. Revisión del Diagrama de Clases

1.1 Introducción

En esta sección se presenta la revisión y actualización del diagrama de clases del Sistema Informático de Gestión de Servicios Médicos (S.I.G.S.M.), tomando como referencia los avances realizados desde la segunda entrega y la estructura actual del código fuente.

El objetivo de esta revisión es reflejar los cambios introducidos durante el desarrollo, principalmente en la organización del backend, donde se incorporaron nuevas clases y se distribuyeron las responsabilidades entre diferentes servicios especializados.

El diagrama actualizado permitirá visualizar las clases implementadas, sus principales atributos y métodos, así como las relaciones y dependencias existentes entre ellas, proporcionando una representación más precisa de la arquitectura del sistema.

1.2 Cambios respecto a la segunda entrega

En la segunda entrega del proyecto, el diagrama de clases estaba compuesto únicamente por dos clases: Database, encargada de gestionar la conexión con la base de datos, y Traslado, destinada a representar la lógica asociada a la gestión de traslados.

A medida que avanzó el desarrollo, se identificó la necesidad de reorganizar el backend para evitar concentrar demasiadas responsabilidades en una misma clase y facilitar el mantenimiento del código. Como resultado, se adoptó una estructura

más modular, incorporando clases de servicio especializadas en operaciones concretas.

Esta reorganización se refleja principalmente en el módulo de traslados, donde se separaron las responsabilidades de creación, consulta, asignación de recursos, disponibilidad, actualización de estados y cancelación de solicitudes. Asimismo, se incorporaron clases para gestionar operaciones relacionadas con las encuestas y la consulta de funcionarios.

Cabe destacar que esta separación no implicó convertir todas las funcionalidades del sistema en clases. La API continúa utilizando archivos independientes, funciones, endpoints y handlers para procesar las solicitudes HTTP, mientras que las clases se emplean en aquellos componentes donde resulta conveniente encapsular la lógica de negocio y el acceso a los datos.

De esta manera, el diagrama de clases actualizado representa la evolución de la estructura interna de S.I.G.S.M., mostrando una organización más clara de las responsabilidades y una mayor facilidad para realizar modificaciones, pruebas y futuras ampliaciones.

1.3 Identificación de las clases implementadas

A partir de la revisión del código fuente del backend de S.I.G.S.M., se identificaron las principales clases utilizadas para organizar el acceso a los datos y la lógica de negocio del sistema.

Estas clases se encuentran distribuidas principalmente entre los directorios config y services de la API. Su organización responde a la necesidad de separar responsabilidades, evitando concentrar las operaciones de los distintos módulos en una única clase.

A continuación, se presentan las clases identificadas, agrupadas según su funcionalidad.

1.3.1 Configuración y acceso a datos

Database: encargada de establecer y proporcionar la conexión con MariaDB mediante PDO. Centraliza la configuración de acceso a la base de datos y permite reutilizar la conexión durante la ejecución de una solicitud.

1.3.2 Módulo de gestión de traslados

- **TransferCreateService**: gestiona la creación de nuevas solicitudes de traslado, incluyendo las validaciones de los tipos de traslado, elementos y recorridos permitidos.
- **TransferQueryService**: realiza las consultas de traslados activos, finalizados e individuales, incluyendo la recuperación de información histórica.
- **TransferAssignmentService**: administra la asignación de vehículos y funcionarios a los traslados, verificando la compatibilidad de recursos y evitando asignaciones que presenten conflictos de disponibilidad.
- **TransferResourceService**: consulta la disponibilidad de vehículos, conductores y enfermeros según los horarios establecidos para cada traslado.
- **TransferStateService**: controla las transiciones entre los diferentes estados de un traslado, comprueba las condiciones necesarias para avanzar y registra los cambios realizados.
- **TransferDeleteService**: gestiona la cancelación de solicitudes de traslado, aplicando las comprobaciones necesarias para determinar si la operación está permitida.

1.3.3 Módulo de encuestas

- **SurveyCreateService**: gestiona la creación de nuevas encuestas y el almacenamiento de su información inicial.
- **SurveyQueryService**: permite consultar las encuestas registradas en el sistema, tanto de forma general como individual.
- **SurveyUpdateService**: administra las modificaciones de las encuestas existentes y realiza las validaciones necesarias para su publicación.
- **SurveyResultsService**: obtiene los resultados de las encuestas a partir de las respuestas almacenadas en la base de datos.

1.3.4 Administración de funcionarios

- **UserQueryService**: centraliza las consultas de funcionarios registrados en el sistema, permitiendo recuperar la información necesaria para su administración.

1.3.5 Organización de las clases

Las clases identificadas siguen una organización basada en la separación de responsabilidades. Cada servicio se especializa en un conjunto concreto de operaciones, mientras que la conexión con la base de datos se proporciona mediante la clase Database.

Las clases de servicio reciben una conexión PDO a través de sus constructores, lo que permite utilizar un mecanismo común de acceso a los datos sin incorporar directamente la configuración de conexión en cada servicio.

Por otra parte, determinadas funcionalidades, como la gestión documental, la autenticación y diversas operaciones administrativas, se implementan mediante funciones y archivos PHP independientes. Estos componentes no se representan

como clases, ya que no forman parte de la estructura orientada a objetos del backend.

Esta organización permite mantener una arquitectura modular, facilitando la comprensión del código, la localización de errores y la incorporación de nuevas funcionalidades sin afectar innecesariamente a otros componentes del sistema.

1.4 Relaciones y dependencias entre clases

Las relaciones entre las clases de S.I.G.S.M. se basan principalmente en el uso compartido de la conexión a la base de datos y en la separación de responsabilidades entre los distintos servicios del backend.

A diferencia de una arquitectura basada en herencia, donde unas clases extienden el comportamiento de otras, el sistema utiliza principalmente relaciones de dependencia, permitiendo que cada clase se encargue de operaciones específicas sin necesidad de establecer jerarquías entre ellas.

1.4.1 Relación con la clase Database

La clase Database centraliza la creación y reutilización de la conexión con MariaDB mediante PDO (PHP Data Objects).

Esta clase implementa un mecanismo de conexión compartida a través del método estático getConnection(), que devuelve una instancia de PDO. De esta manera, los diferentes componentes de la API pueden acceder a la base de datos sin tener que implementar su propia configuración de conexión.

Los servicios reciben dicha conexión mediante sus constructores, almacenándola en un atributo privado denominado $db.

Esta relación permite separar la configuración de la base de datos de las operaciones realizadas por cada servicio, facilitando el mantenimiento y la modificación de los parámetros de conexión.

1.4.2 Dependencias de los servicios

Las clases de servicio utilizan la conexión PDO recibida para ejecutar las consultas y operaciones necesarias sobre la base de datos.

Aunque varios servicios trabajan con información relacionada, como ocurre con las solicitudes de traslado, cada uno mantiene sus propias responsabilidades y no necesita heredar de otros servicios.

Por ejemplo, TransferCreateService se encarga de registrar nuevas solicitudes, mientras que TransferAssignmentService administra la asignación de recursos y TransferStateService controla las transiciones de estado.

Esta separación reduce el acoplamiento entre los componentes y permite modificar determinadas funcionalidades sin alterar directamente la implementación de las demás.

1.4.3 Relación con los handlers y endpoints

Además de las dependencias entre clases, la arquitectura del backend establece una separación entre los endpoints, los handlers y los servicios.

Los endpoints reciben las solicitudes HTTP y determinan qué operación debe ejecutarse. Los handlers se encargan de procesar los parámetros recibidos, realizar las validaciones correspondientes y utilizar los servicios necesarios para completar la operación.

Cuando una operación requiere un servicio, el handler obtiene la conexión mediante Database::getConnection() y la proporciona al constructor de la clase correspondiente.

Los handlers y endpoints no se representan como clases dentro del diagrama UML, debido a que están implementados principalmente mediante funciones y archivos PHP independientes.

1.4.4 Representación de las relaciones en el diagrama

En el diagrama de clases actualizado se representarán las dependencias asociadas al acceso a los datos, mostrando cómo los servicios utilizan objetos PDO y cómo la clase Database proporciona la conexión.

No se incluirán relaciones de herencia, composición o agregación entre los servicios, ya que estas no forman parte de la implementación actual.

De esta manera, el diagrama reflejará la estructura real del backend, destacando la independencia de los servicios y la centralización del acceso a la base de datos.

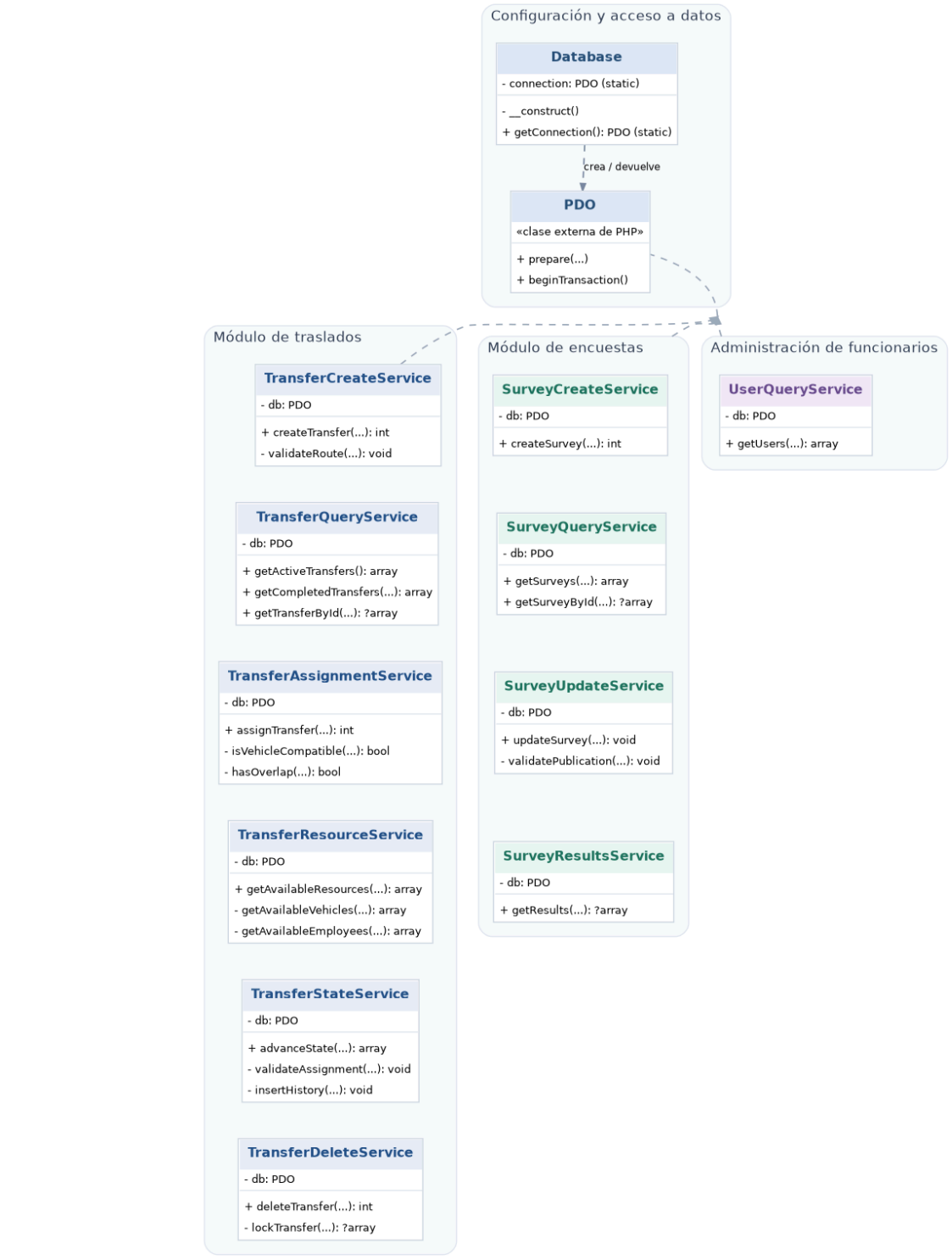
1.5 Diagrama de clases actualizado

A continuación, se presenta el diagrama de clases actualizado de S.I.G.S.M., elaborado a partir de la estructura implementada en el backend del sistema.

Debido a la cantidad de clases y métodos incorporados durante el desarrollo, se optó por realizar una representación simplificada, incluyendo los atributos y métodos más representativos de cada clase, así como sus principales dependencias.

Esta decisión busca facilitar la lectura e interpretación del diagrama, evitando una representación excesivamente extensa que dificulte comprender la arquitectura general del sistema. Por lo tanto, el diagrama no incluye la totalidad de los métodos implementados, los cuales pueden consultarse directamente en el código fuente disponible en el repositorio de GitHub.

Las clases se encuentran agrupadas según sus responsabilidades, permitiendo visualizar la organización modular del backend y la separación de las distintas funcionalidades del sistema.



Leyenda: + público | - privado | flecha discontinua: dependencia PDO (agrupada por módulo) | (...) parámetros abreviados

1.6 Justificación de la estructura modular

Durante el desarrollo de S.I.G.S.M. se optó por organizar el backend mediante una arquitectura modular, separando las distintas funcionalidades del sistema en componentes con responsabilidades específicas.

Esta decisión surgió principalmente de la necesidad de evitar clases y archivos excesivamente extensos, que concentraran numerosas operaciones y dificultaran su comprensión y mantenimiento. En lugar de implementar toda la lógica de un módulo dentro de una única clase, se decidió distribuirla entre diferentes servicios especializados.

Un ejemplo de esta organización se encuentra en el módulo de traslados, donde las operaciones de creación, consulta, asignación de recursos, verificación de disponibilidad, actualización de estados y cancelación se implementaron en clases independientes. De esta manera, cada servicio se encarga de una parte concreta del funcionamiento del módulo.

La separación de responsabilidades también se aplicó al módulo de encuestas, mediante clases dedicadas a su creación, consulta, modificación y obtención de resultados.

Asimismo, se centralizó el acceso a la base de datos mediante la clase Database, mientras que los servicios reciben una conexión PDO a través de sus constructores. Esto evita duplicar la configuración de conexión y permite mantener separada la lógica de negocio de los aspectos relacionados con la infraestructura.

Por otra parte, se mantuvo una estructura basada en endpoints, handlers, servicios y utilidades, permitiendo separar el procesamiento de las solicitudes HTTP de las operaciones internas del sistema.

Esta organización presenta diferentes ventajas para el desarrollo y mantenimiento de S.I.G.S.M.:

- **Mantenibilidad**: facilita localizar y modificar funcionalidades específicas sin necesidad de intervenir en archivos que contienen operaciones no relacionadas.
- **Reutilización**: permite utilizar componentes y funciones comunes desde diferentes partes de la aplicación.
- **Escalabilidad**: facilita incorporar nuevas funcionalidades o ampliar las existentes sin reorganizar completamente el backend.
- **Facilidad de pruebas**: permite comprobar el funcionamiento de servicios y operaciones de manera más independiente.
- **Organización del código**: mejora la legibilidad y facilita la distribución de tareas entre los integrantes del equipo.

En conclusión, la adopción de una estructura modular permitió organizar progresivamente el backend de S.I.G.S.M. conforme aumentó su complejidad. Esta decisión responde a las necesidades reales del proyecto y establece una base más ordenada para su mantenimiento, corrección de errores y futuras ampliaciones.

2. Plan de Testing

2.1 Introducción y objetivos

El presente plan de testing tiene como finalidad documentar las pruebas realizadas para verificar el correcto funcionamiento del Sistema Informático de Gestión de Servicios Médicos (S.I.G.S.M.), comprobando que las funcionalidades desarrolladas cumplen con los requerimientos establecidos durante el análisis del proyecto.

Las pruebas se centraron principalmente en la validación funcional de los distintos módulos del sistema, incluyendo la autenticación y administración de funcionarios, la gestión documental, las encuestas y la gestión y seguimiento de traslados.

Para ello, se elaboraron casos de prueba que contemplan tanto situaciones de funcionamiento esperado como escenarios alternativos y condiciones de error. Asimismo, se utilizaron tablas de decisión para representar el comportamiento del sistema ante diferentes condiciones y reglas de negocio.

Los principales objetivos del plan de testing fueron:

- **Verificar el cumplimiento de los requerimientos funcionales**: comprobar que las operaciones implementadas responden correctamente a las necesidades establecidas para el sistema.
- **Detectar errores de funcionamiento**: identificar posibles fallos en el procesamiento de solicitudes, validación de datos y ejecución de operaciones.
- **Validar las reglas de negocio**: comprobar que se respetan las restricciones definidas, especialmente en la gestión de permisos, asignación de recursos y transiciones de estado.
- **Comprobar la integración entre componentes**: verificar la comunicación entre el frontend, la API y la base de datos.
- **Evaluar el manejo de situaciones excepcionales**: comprobar que el sistema responde adecuadamente ante datos incorrectos, solicitudes no autorizadas o conflictos durante las operaciones.
- **Registrar los resultados obtenidos**: documentar las pruebas realizadas y sus resultados para contar con evidencia del funcionamiento del sistema.

Mediante este proceso se buscó evaluar la confiabilidad de S.I.G.S.M. e identificar posibles problemas antes de su entrega final.

2.2 Alcance y metodología de pruebas

El proceso de testing de S.I.G.S.M. contempló la verificación de las funcionalidades implementadas en los diferentes módulos del sistema, incluyendo tanto las operaciones realizadas desde la interfaz web como el funcionamiento de la API y su interacción con la base de datos.

Durante el desarrollo se realizaron pruebas de forma progresiva, a medida que se incorporaban nuevas funcionalidades. Estas permitieron comprobar el comportamiento de los componentes, detectar errores y realizar las correcciones necesarias antes de continuar con las siguientes etapas.

La metodología utilizada se basó principalmente en pruebas funcionales de caja negra, complementadas con pruebas de integración y pruebas exploratorias de la interfaz.

Se contemplaron los siguientes tipos de pruebas:

- Pruebas funcionales: comprobación de que las operaciones del sistema producen los resultados esperados ante diferentes entradas y condiciones.
- Pruebas de integración: verificación de la comunicación entre el frontend, los endpoints de la API y la base de datos.
- Pruebas de validación: comprobación del comportamiento del sistema ante datos incompletos, incorrectos o que incumplen las reglas de negocio.
- Pruebas de autorización: verificación de que las funcionalidades restringidas solo pueden ser utilizadas por funcionarios con los permisos correspondientes.
- Pruebas exploratorias: utilización de la aplicación para identificar posibles comportamientos inesperados o dificultades de uso.
- Pruebas de regresión: repetición de comprobaciones sobre funcionalidades previamente desarrolladas después de introducir modificaciones.

Para organizar y documentar las comprobaciones, se utilizó una planilla compartida en Google Sheets, donde se registraron 36 casos de prueba correspondientes a los distintos módulos del sistema.

Cada caso incluye un identificador, la funcionalidad evaluada, las condiciones necesarias para su ejecución, los pasos realizados, el resultado esperado, el tipo de prueba, su prioridad, el estado, el resultado obtenido, el responsable y la fecha de ejecución.

Las pruebas se realizaron tomando como referencia las funcionalidades y casos de uso definidos durante el desarrollo, contemplando tanto operaciones correctas como situaciones alternativas y de error.

El alcance del testing se limitó a los componentes desarrollados por el equipo. No se incluyeron pruebas de integración con sistemas internos del Hospital de Clínicas cuya documentación o acceso no fueron proporcionados por la institución.

2.3 Entorno y herramientas utilizadas

Las pruebas se realizaron utilizando el entorno de desarrollo implementado para S.I.G.S.M., compuesto por las tecnologías empleadas en el frontend, backend y base de datos.

El entorno utilizado incluye:

- **Frontend**: aplicación desarrollada con React, Vite y Material UI, ejecutada desde un navegador web.
- **Backend**: API REST desarrollada en PHP 8.2.
- **Base de datos:** MariaDB, utilizada para almacenar la información de funcionarios, documentos, encuestas y traslados.

- **Servidor**: entorno Linux basado en Fedora Server, ejecutado mediante una máquina virtual.
- **XAMPP**: utilizado para proporcionar los servicios necesarios para la ejecución del backend y el acceso a la base de datos.
- **Navegador web**: utilizado para realizar las pruebas funcionales y comprobar el comportamiento de la interfaz.
- **Herramientas de terminal**: utilizadas para enviar solicitudes HTTP a los endpoints de la API y verificar sus respuestas.
- **Google Sheets**: utilizado para organizar los casos de prueba y registrar los resultados obtenidos.

Las comprobaciones incluyeron operaciones realizadas directamente desde la interfaz y solicitudes a la API, permitiendo verificar tanto el comportamiento visible para el usuario como las respuestas generadas por el backend.

Asimismo, cuando fue necesario, se consultó la información almacenada en la base de datos para comprobar que las operaciones se ejecutaran correctamente.

2.4 Pruebas funcionales

Las pruebas funcionales tuvieron como objetivo comprobar que las operaciones implementadas en S.I.G.S.M. produjeran los resultados esperados, de acuerdo con las reglas y requerimientos establecidos para cada módulo.

Se ejecutaron un total de 36 casos de prueba, distribuidos entre las principales funcionalidades del sistema.

2.4.1 Autenticación y gestión de funcionarios

Se realizaron pruebas relacionadas con el inicio y cierre de sesión, la validación de credenciales y las restricciones de acceso a funcionalidades administrativas.

Entre las situaciones evaluadas se encuentran el inicio de sesión con credenciales válidas, el rechazo de contraseñas incorrectas, el acceso de funcionarios inactivos y la protección de rutas que requieren autenticación.

También se comprobó la consulta de funcionarios, las restricciones según los roles asignados y el registro de solicitudes de acceso.

En total, se ejecutaron 8 casos de prueba correspondientes a autenticación y administración de funcionarios, identificados con los códigos AUT-01 a AUT-05 y USR-01 a USR-03.

2.4.2 Gestión documental

Las pruebas del módulo de documentación se centraron en la administración de categorías y documentos, así como en la validación de los archivos y las restricciones de acceso.

Se comprobaron operaciones como la creación de categorías, el rechazo de datos incompletos, la carga de documentos PDF, la edición de su información, la desactivación y la eliminación de documentos.

Asimismo, se verificó el acceso a documentos publicados y las restricciones correspondientes a aquellos que se encuentran inactivos.

En total, se ejecutaron 8 casos de prueba, identificados con los códigos DOC-01 a DOC-08.

2.4.3 Gestión de encuestas

Las pruebas del módulo de encuestas se orientaron a comprobar las operaciones de creación, publicación, consulta, respuesta y visualización de resultados.

Se verificó el registro de nuevas encuestas, las validaciones necesarias para su publicación y el comportamiento de los formularios ante respuestas completas e incompletas.

También se comprobó la consulta de encuestas publicadas y la consistencia de los resultados obtenidos a partir de las respuestas registradas.

En total, se ejecutaron 6 casos de prueba, identificados con los códigos ENC-01 a ENC-06.

2.4.4 Gestión y seguimiento de traslados

El módulo de traslados recibió especial atención debido a la cantidad de reglas de negocio involucradas en su funcionamiento.

Las pruebas incluyeron la creación de solicitudes, la validación de fechas y recorridos, la consulta de recursos disponibles y la asignación de vehículos y funcionarios.

También se verificó que el sistema impidiera asignaciones incompatibles o que presentaran conflictos de disponibilidad.

En cuanto al seguimiento, se comprobaron las transiciones válidas entre estados, el rechazo de cambios fuera de secuencia y la detección de conflictos cuando se intenta modificar un traslado utilizando una versión desactualizada.

Finalmente, se evaluaron la cancelación de solicitudes y la consulta del historial de traslados.

En total, se ejecutaron 11 casos de prueba, identificados con los códigos TRA-01 a TRA-11.

Adicionalmente, se realizaron 3 pruebas exploratorias de la interfaz, identificadas como UI-01 a UI-03, enfocadas en la navegación, la adaptación a diferentes tamaños de pantalla y la presentación de mensajes de error.

2.5 Tablas de decisión

Las tablas de decisión permiten representar el comportamiento esperado del sistema ante distintas combinaciones de condiciones, facilitando la comprensión y verificación de las reglas de negocio.

Se seleccionaron funcionalidades donde la ejecución de una operación depende del cumplimiento de múltiples condiciones.

Tabla de decisión 1. Acceso a funcionalidades restringidas

Sesión válida	Rol autorizado	Resultado
Sí	Sí	Acceso permitido
Sí	No	Acceso denegado

No	Sí	Acceso denegado
No	No	Acceso denegado

Esta tabla representa las condiciones necesarias para acceder a una funcionalidad protegida. El usuario debe encontrarse autenticado y disponer del rol requerido para ejecutar la operación.

Tabla de decisión 2. Asignación de recursos a un traslado

Recursos disponibles	Recursos compatibles	Versión válida	Resultado
Sí	Sí	Sí	Asignación permitida
No	Sí	Sí	Asignación rechazada
Sí	No	Sí	Asignación rechazada
Sí	Sí	No	Conflicto de versión
No	No	Sí	Asignación rechazada

Esta tabla representa algunas de las condiciones principales que intervienen en la asignación de vehículos y funcionarios. Además de estas condiciones, el sistema realiza otras comprobaciones relacionadas con el estado del traslado y los permisos del funcionario.

Tabla de decisión 3. Actualización del estado de un traslado

Transición válida	Condiciones requeridas cumplidas	Versión válida	Resultado
Sí	Sí	Sí	Estado actualizado
No	Sí	Sí	Cambio rechazado
Sí	No	Sí	Cambio rechazado
Sí	Sí	No	Conflicto de versión

Esta tabla representa las condiciones generales para avanzar un traslado al siguiente estado. El sistema debe verificar que la transición corresponda a la secuencia definida, que se cumplan las condiciones necesarias para realizarla y que la versión del registro sea válida.

Las tablas presentadas permiten resumir las principales reglas de decisión evaluadas durante el testing. No representan todas las combinaciones posibles del sistema, sino aquellas consideradas más relevantes para ilustrar su funcionamiento.

2.6 Casos de prueba basados en casos de uso

Los casos de prueba se elaboraron tomando como referencia las funcionalidades y escenarios contemplados en los casos de uso del sistema.

Cada prueba permitió comprobar un comportamiento concreto mediante condiciones de entrada, pasos de ejecución y un resultado esperado.

El registro completo de los 36 casos de prueba se encuentra disponible en la planilla de testing del proyecto, utilizada como documento complementario de esta entrega.

2.7 Registro de resultados e incidencias

Los resultados del testing se documentaron en una planilla compartida de Google Sheets, permitiendo mantener un registro organizado de las pruebas realizadas sobre el sistema.

La ejecución de los casos registrados estuvo a cargo de Martín Cabrera, con fecha de ejecución del 5 de noviembre de 2026.

A continuación, se presenta un resumen de los resultados obtenidos:

Módulo	Casos ejecutados	Resultados satisfactorios
Autenticación	5	5
Administración de funcionarios	3	3
Gestión documental	8	8
Encuestas	6	6
Traslados	11	11
Interfaz de usuario	3	3
Total	**36**	**36**

De acuerdo con el registro realizado, los 36 casos de prueba fueron ejecutados y sus resultados coincidieron con los comportamientos esperados, obteniéndose un porcentaje de cumplimiento del 100 % para los escenarios evaluados.

No se registraron incidencias en esta ejecución del plan de testing.

Cabe destacar que estos resultados corresponden a los casos definidos y ejecutados, por lo que no implican necesariamente la ausencia de errores en funcionalidades o escenarios que no hayan sido contemplados durante las pruebas.

Plan de testing de S.I.G.S.M.: Plan testing - Hojas de cálculo de Google

2.8 Conclusiones del testing

La ejecución del plan de testing permitió comprobar el funcionamiento de las principales funcionalidades desarrolladas en S.I.G.S.M., incluyendo la autenticación, administración de funcionarios, gestión documental, encuestas y gestión de traslados.

Los resultados obtenidos fueron satisfactorios, alcanzándose el cumplimiento de los resultados esperados en los 36 casos de prueba ejecutados.

Las pruebas permitieron verificar tanto operaciones habituales como situaciones alternativas y condiciones de error, especialmente en aquellos módulos donde existen restricciones de acceso, validaciones de datos y reglas de negocio.

En particular, las comprobaciones realizadas sobre el módulo de traslados permitieron evaluar aspectos relevantes como la disponibilidad de recursos, la prevención de asignaciones incompatibles, las transiciones de estado y el control de versiones de los registros.

Asimismo, el uso de una planilla compartida permitió organizar y centralizar los resultados, facilitando su consulta y proporcionando un respaldo documental de las pruebas realizadas.

En conclusión, los resultados del testing aportan evidencia favorable sobre el funcionamiento de las funcionalidades evaluadas y respaldan el grado de cumplimiento alcanzado durante el desarrollo del proyecto.

No obstante, se reconoce que será necesario continuar realizando pruebas ante futuras modificaciones o ampliaciones del sistema, especialmente cuando se incorporen integraciones con servicios externos o nuevas funcionalidades.

3. Comparación de Métricas de Tamaño y Función

3.1 Introducción y objetivo de la comparación

En la segunda entrega de S.I.G.S.M. se realizó una estimación del tamaño funcional del sistema mediante el método de Puntos de Función, considerando los requerimientos y las funcionalidades previstas en aquella etapa del proyecto.

Desde entonces, el desarrollo avanzó considerablemente, incorporando nuevas operaciones, ampliando funcionalidades existentes y reorganizando la estructura interna del código.

El objetivo de esta sección es comparar las métricas obtenidas durante la segunda entrega con una estimación actualizada, tomando como referencia las funcionalidades implementadas y el estado del código fuente.

Para ello, se analizarán dos aspectos:

- **Métricas de tamaño**: cantidad de archivos de código fuente y tamaño de estos archivos, utilizados para observar el crecimiento de la implementación.

- **Métricas de función**: cantidad y complejidad de las operaciones ofrecidas por el sistema, expresadas mediante Puntos de Función Sin Ajustar (PFSA) y Puntos de Función Ajustados (PFA).

Esta comparación permitirá identificar las principales variaciones entre ambas etapas y evaluar cómo evolucionó la complejidad del sistema durante su desarrollo.

3.2 Métricas obtenidas en la segunda entrega

En la segunda entrega se utilizó el método de Puntos de Función, clasificando las funcionalidades identificadas en cinco categorías: entradas externas, salidas externas, consultas externas, archivos lógicos internos (ALI) y archivos de interfaz externos (AIE).

Cada componente fue clasificado según su complejidad baja, media o alta, asignándole el peso correspondiente.

Los resultados obtenidos fueron los siguientes:

Tipo de función	Baja	Media	Alta	Total
Entrada	10	8	5	23
Salida	6	5	2	13
Consulta	5	5	0	10
ALI	3	5	1	9
AIE	0	1	0	1
Total	24	24	8	56

A partir de esta clasificación, se obtuvieron los siguientes valores:

- Puntos de Función Sin Ajustar (PFSA): 283.
- Suma de los factores de influencia (Fi): 45.
- Factor de Ajuste (FA): 1,10.
- Puntos de Función Ajustados (PFA): 311,3.

El cálculo final fue:

PFA = 283 × 1,10 = 311,3

Estos valores representaban una estimación del tamaño funcional previsto para S.I.G.S.M. en aquella etapa.

Cabe destacar que el cálculo incluía funcionalidades que todavía no se encontraban implementadas, como la consulta de información de pacientes mediante un sistema externo proporcionado por el Hospital de Clínicas.

3.3 Actualización de las métricas de tamaño

Para evaluar el crecimiento del código fuente se realizó una comparación entre dos versiones del repositorio de GitHub:

Segunda entrega: versión registrada el 1 de septiembre de 2026, inmediatamente anterior a la fecha establecida para dicha entrega.

Tercera entrega: estado de la rama main consultado el 5 de noviembre de 2026.

El análisis se limitó a los archivos PHP del backend y a los archivos JavaScript y JSX del frontend.

No se incluyeron dependencias externas, archivos de configuración, documentación, imágenes, traducciones ni scripts SQL. Los archivos de pruebas escritos en los lenguajes indicados sí forman parte del recuento.

Se obtuvieron los siguientes resultados:

Componente	Segunda entrega	Tercera entrega	Variación
Archivos PHP	12	70	+58
Archivos JavaScript/JSX	30	74	+44
Total de archivos	42	144	+102
Tamaño del código	128 KiB	718 KiB	+590 KiB

Los resultados muestran un incremento del 242,9 % en la cantidad de archivos de código fuente y del 461 % en el tamaño total de estos archivos.

Este crecimiento se relaciona principalmente con la incorporación de nuevas funcionalidades y con la reorganización del backend mediante endpoints, handlers y clases de servicio especializadas.

En el frontend también se observa un incremento significativo, asociado al desarrollo de nuevas pantallas, componentes reutilizables, formularios, diálogos y funciones de comunicación con la API.

Es importante señalar que el tamaño del código no representa directamente su calidad ni su complejidad funcional. Una arquitectura modular puede incrementar la cantidad de archivos sin que esto implique necesariamente un aumento equivalente en las funcionalidades ofrecidas.

Por este motivo, las métricas de tamaño se complementan con el análisis de Puntos de Función.

3.4 Actualización de las métricas de función

Para actualizar las métricas funcionales se revisaron las operaciones disponibles en los distintos módulos de S.I.G.S.M., manteniendo las categorías y los pesos utilizados en la segunda entrega.

La revisión permitió identificar funcionalidades incorporadas posteriormente, entre las que se destacan:

- Administración de solicitudes de acceso y asignación de roles a funcionarios.
- Eliminación y reemplazo de archivos en el módulo documental.
- Creación, modificación y reordenamiento de preguntas de encuestas.
- Asignación de vehículos y funcionarios a traslados.
- Consulta de disponibilidad de recursos según intervalos horarios.
- Cancelación de solicitudes y consulta del historial de traslados.
- Incorporación de filtros y consultas más específicas.

También se revisaron las funcionalidades previstas originalmente que no llegaron a implementarse o que fueron reorganizadas durante el desarrollo.

En particular, no se contabilizó la consulta de pacientes mediante una API externa, debido a que el Hospital de Clínicas no proporcionó el acceso necesario para realizar dicha integración.

La nueva estimación identifica 62 componentes funcionales, distribuidos en 29 entradas, 15 salidas, 7 consultas, 11 archivos lógicos internos y ningún archivo de interfaz externo.

El recuento considera operaciones reconocibles desde el punto de vista del usuario, por lo que no equivale directamente a la cantidad de endpoints, funciones PHP o tablas de la base de datos.

3.4.1 Puntos de función sin ajustar (PFSA)

Para obtener los nuevos PFSA se mantuvieron los pesos utilizados en la segunda entrega:

Tipo de función	Baja	Media	Alta
Entrada	x3	x4	x6
Salida	x4	x5	x7
Consulta	x3	x4	x6
ALI	x7	x10	x15
AIE	x5	x7	x10

A partir del análisis actualizado se obtuvo la siguiente clasificación:

Tipo de función	Baja	Media	Alta	Total
Entradas	10	12	7	29
Salidas	6	7	2	15
Consultas	2	5	0	7
ALI	3	7	1	11
AIE	0	0	0	0
Total	21	31	10	62

El incremento de las entradas se relaciona principalmente con las nuevas operaciones de administración, gestión de encuestas y asignación de recursos.

En el caso de las salidas, se incorporaron nuevas formas de presentar información, como los recursos disponibles, los historiales de traslado y los listados administrativos.

Por su parte, la reducción de consultas responde a la revisión de las operaciones originalmente estimadas, evitando contabilizar por separado funcionalidades que actualmente se encuentran agrupadas en una misma operación lógica.

Los archivos lógicos internos se actualizaron considerando los principales grupos de información administrados por el sistema, incluyendo funcionarios, roles, categorías, documentos, encuestas, preguntas y opciones, respuestas, traslados, historial de estados, vehículos y catálogos relacionados.

No se contabilizaron archivos de interfaz externos, ya que la integración prevista con el sistema de pacientes del hospital continúa pendiente.

Aplicando los pesos correspondientes:

Tipo de función	Cálculo	PFSA
Entrada	(10 × 3) + (12 × 4) + (7 × 6)	120
Salida	(6 × 4) + (7 × 5) + (2 × 7)	73
Consulta	(2 × 3) + (5 × 4)	26
ALI	(3 × 7) + (7 × 10) + (1 × 15)	106

AIE	0	0
Total		**325**

Por lo tanto, el nuevo valor estimado es:

PFSA = 325

En comparación con la segunda entrega, se observa un aumento de 42 puntos de función sin ajustar.

3.4.2 Factor de ajuste (FA)

Para actualizar el Factor de Ajuste se revisaron las 14 características generales evaluadas en la segunda entrega, considerando los cambios introducidos durante el desarrollo.

Se mantuvieron los valores correspondientes a características cuyo grado de influencia no presentó variaciones relevantes.

Las principales modificaciones fueron:

- Comunicación con otros sistemas: se redujo de 4 a 0, debido a que la integración externa prevista con los sistemas del hospital no pudo implementarse.
- Reutilización: se incrementó de 3 a 4, considerando la incorporación de servicios especializados, componentes reutilizables y una organización más modular.
- Facilidades de instalación y conversión: se incrementó de 2 a 4, debido a la incorporación de una configuración de despliegue mediante Docker y documentación específica para la instalación del sistema.

La evaluación actualizada se presenta a continuación:

N.°	Característica evaluada	Segunda entrega	Tercera entrega
1	Recuperación y copias de seguridad	4	4
2	Comunicación con otros sistemas	4	0
3	Procesamiento distribuido	3	3
4	Rendimiento	3	3
5	Utilización de infraestructura existente	2	2
6	Ingreso de datos en línea	5	5
7	Ingreso de datos mediante varios pasos	3	3
8	Actualización directa de datos principales	4	4
9	Complejidad de entradas, salidas y consultas	3	3
10	Complejidad del procesamiento interno	4	4
11	Reutilización	3	4
12	Facilidades de instalación y conversión	2	4
13	Instalación en diferentes organizaciones	1	1

14	Facilidad de modificación y uso	4	4
Total		**45**	**44**

Para calcular el Factor de Ajuste se utiliza la misma fórmula aplicada anteriormente:

FA = 0,65 + (0,01 × Fi)

Sustituyendo los valores:

FA = 0,65 + (0,01 × 44)

FA = 0,65 + 0,44

FA = 1,09

El nuevo Factor de Ajuste es, por lo tanto, 1,09.

Esta variación refleja una ligera reducción en el valor de ajuste general, principalmente por la ausencia de integración con sistemas externos, parcialmente compensada por las mejoras realizadas en la organización y el despliegue de la aplicación.

3.4.3 Puntos de función ajustados (PFA)

Una vez obtenidos los nuevos PFSA y el Factor de Ajuste, se procede a calcular los Puntos de Función Ajustados.

Se utiliza la fórmula:

PFA = PFSA × FA

Los valores actualizados son:

PFSA = 325

FA = 1,09

Por lo tanto:

PFA = 325 × 1,09

PFA = 354,25

El tamaño funcional estimado de S.I.G.S.M. para la tercera entrega es de aproximadamente 354,25 Puntos de Función Ajustados.

Este valor representa una actualización de la estimación inicial, teniendo en cuenta las funcionalidades identificadas en el código y las características generales del sistema en su etapa actual.

3.5 Comparación de resultados entre entregas

A continuación, se presenta una comparación general de las métricas funcionales obtenidas en ambas etapas del proyecto.

Métrica	Segunda entrega	Tercera entrega	Variación
Entradas	23	29	+6
Salidas	13	15	+2

Consultas	10	7	-3
ALI	9	11	+2
AIE	1	0	-1
Componentes funcionales	56	62	+6
PFSA	283	325	+42
FA	1,10	1,09	-0,01
PFA	311,3	354,25	+42,95

El tamaño funcional ajustado presenta un incremento estimado del 13,8 % respecto a la segunda entrega.

Aunque el crecimiento de la cantidad de archivos de código fuente fue considerablemente mayor, esto no se traduce en un incremento equivalente de los Puntos de Función.

Esta diferencia se debe a que ambas métricas evalúan aspectos distintos: el tamaño del código refleja la extensión física de la implementación, mientras que los Puntos de Función representan las operaciones y los grupos de información reconocibles por el usuario.

3.6 Comparación de la estimación del esfuerzo

En la segunda entrega se realizó una estimación teórica del esfuerzo necesario para desarrollar S.I.G.S.M., tomando como referencia los Puntos de Función Ajustados obtenidos.

Para mantener la coherencia entre ambas entregas, se utilizará la misma fórmula de estimación:

Esfuerzo = (PFA / 150) × PFA^0,4

Donde:

- **PFA**: representa los Puntos de Función Ajustados del sistema.
- **150**: corresponde a la productividad de referencia utilizada en la estimación original.
- **0,4:** es el exponente aplicado para considerar las economías de escala.

Estimación de la segunda entrega

En la segunda entrega se obtuvo un tamaño funcional de 311,3 PFA.

Esfuerzo = (311,3 / 150) × 311,3^0,4

Esfuerzo = 20,62 semanas

Este resultado representaba aproximadamente 4,8 meses de trabajo.

Estimación de la tercera entrega

Para la tercera entrega se obtuvo un tamaño funcional actualizado de 354,25 PFA.

Aplicando la misma fórmula:

Esfuerzo = (354,25 / 150) × 354,25^0,4

Esfuerzo = 24,71 semanas

Este resultado representa aproximadamente 5,7 meses de trabajo.

Comparación de resultados

Métrica	Segunda entrega	Tercera entrega	Variación
Puntos de Función Ajustados	311,3	354,25	+13,8%
Esfuerzo estimado (semanas)	20,62	24,71	+4,09
Esfuerzo estimado (meses)	4,8	5,7	+0,9

La comparación muestra un incremento aproximado del 19,8 % en el esfuerzo estimado, relacionado con el aumento del tamaño funcional del sistema.

Esta variación responde principalmente a la incorporación de nuevas operaciones y a la ampliación de funcionalidades que no se encontraban completamente definidas durante la segunda entrega.

Cabe destacar que el resultado representa una estimación teórica del tiempo total requerido para desarrollar el proyecto, y no el tiempo adicional necesario para finalizarlo a partir de la tercera entrega.

Asimismo, el esfuerzo estimado no necesariamente coincide con el tiempo real empleado, ya que este depende de factores como la experiencia de los integrantes, la distribución de tareas, el aprendizaje de nuevas tecnologías y las dificultades surgidas durante el desarrollo.

En conclusión, la actualización de esta métrica permite observar cómo el crecimiento funcional de S.I.G.S.M. repercute sobre el esfuerzo de desarrollo previsto, evidenciando la importancia de revisar las estimaciones iniciales conforme avanza el proyecto.

3.7 Análisis de las variaciones

Las diferencias observadas entre ambas entregas responden principalmente a la evolución del proyecto desde una etapa de diseño y estimación hacia una implementación funcional más completa.

Durante la segunda entrega, parte de las funcionalidades se encontraban previstas en los requerimientos y casos de uso, pero todavía no habían sido desarrolladas. En la tercera entrega, gran parte de esas operaciones ya cuenta con una implementación concreta, lo que permite realizar una estimación más ajustada al estado actual del sistema.

El incremento del tamaño del código se explica principalmente por la incorporación de nuevas interfaces, validaciones y servicios. También influyó la decisión de separar las responsabilidades del backend en diferentes clases y archivos, favoreciendo su mantenimiento y organización.

En cuanto a las métricas funcionales, se observa un incremento de las entradas y salidas, especialmente en los módulos de encuestas, administración de funcionarios y gestión de traslados. Sin embargo, también se identificaron funcionalidades originalmente previstas que no pudieron incorporarse, como la comunicación con los sistemas de información de pacientes del Hospital de Clínicas.

Como resultado, los Puntos de Función Ajustados aumentaron de 311,3 a 354,25, representando un incremento aproximado del 13,8 % respecto a la segunda entrega.

Este crecimiento también se reflejó en la estimación del esfuerzo de desarrollo, que pasó de 20,62 a 24,71 semanas, suponiendo un incremento de aproximadamente 4,09 semanas, equivalente al 19,8 %.

La diferencia entre ambos porcentajes se debe a que la fórmula utilizada para calcular el esfuerzo no establece una relación lineal con los Puntos de Función, por lo que el aumento del tamaño funcional produce un incremento proporcionalmente mayor en el tiempo estimado.

Estos resultados permiten observar cómo la incorporación de nuevas funcionalidades y el aumento de la complejidad del sistema pueden afectar a las estimaciones realizadas durante las primeras etapas del proyecto.

No obstante, debe considerarse que el esfuerzo calculado representa una estimación teórica y no necesariamente coincide con el tiempo real empleado por el equipo. Factores como el aprendizaje de nuevas tecnologías, las dificultades técnicas, la distribución de tareas y la experiencia de los integrantes pueden influir considerablemente en la duración del desarrollo.

Por este motivo, las variaciones identificadas no deben interpretarse únicamente como un aumento del alcance del proyecto, sino también como una consecuencia de la revisión y actualización de las estimaciones iniciales a partir de las funcionalidades efectivamente desarrolladas.

3.8 Conclusiones

La comparación de métricas permitió observar la evolución de S.I.G.S.M. entre la segunda y la tercera entrega, tanto en términos del tamaño del código fuente como de las funcionalidades implementadas y el esfuerzo estimado para su desarrollo.

El análisis del repositorio mostró un crecimiento significativo en la cantidad y extensión de los archivos, asociado principalmente a la incorporación de nuevas funcionalidades y a la adopción de una arquitectura más modular.

Por su parte, la actualización de los Puntos de Función permitió estimar un tamaño funcional de 354,25 PFA, frente a los 311,3 PFA calculados inicialmente, representando un incremento aproximado del 13,8 %.

Este aumento también repercutió en la estimación del esfuerzo, que pasó de 20,62 a 24,71 semanas, lo que representa un incremento del 19,8 % respecto al cálculo original.

Los resultados evidencian la importancia de revisar las estimaciones iniciales conforme avanza el desarrollo, ya que la incorporación de nuevas funcionalidades, las modificaciones en los requerimientos y las decisiones de implementación pueden alterar tanto el tamaño funcional como el esfuerzo necesario.

Asimismo, la comparación permitió comprobar que el crecimiento del código fuente no implica necesariamente un incremento equivalente en los Puntos de Función, debido a que ambas métricas evalúan aspectos diferentes del sistema.

En conclusión, las métricas obtenidas permiten cuantificar parte de la evolución de S.I.G.S.M. y comprender mejor el impacto de las decisiones tomadas durante el desarrollo. Su actualización proporciona una visión más representativa del alcance

funcional alcanzado y constituye una referencia útil para la planificación y estimación de futuros proyectos.

4. Análisis FODA ponderado

4.1 Introducción

El análisis FODA ponderado permite identificar y evaluar los factores internos y externos que pueden influir en el desarrollo de S.I.G.S.M. y en la capacidad de Empre S.A.S. para llevar adelante el proyecto.

A diferencia del análisis FODA tradicional, esta metodología permite asignar un peso y una calificación a cada factor identificado, obteniendo valores cuantificables que facilitan la interpretación de las fortalezas, oportunidades, debilidades y amenazas.

Para realizar el análisis se utilizaron dos matrices: la Matriz de Evaluación de Factores Internos (EFI) y la Matriz de Evaluación de Factores Externos (EFE).

Los resultados permiten identificar las principales ventajas y limitaciones del equipo, así como las condiciones externas que pueden favorecer o dificultar el desarrollo, mantenimiento y futura ampliación del sistema.

4.2 Criterios de ponderación

Para la elaboración de ambas matrices se asignó a cada factor un peso comprendido entre 0,00 y 1,00 según su importancia relativa. La suma de los pesos de cada matriz debe ser igual a 1,00.

Posteriormente, se estableció una calificación entre 1 y 4, utilizando los siguientes criterios:

Calificación	Factores internos (EFI)	Factores externos (EFE)
1	Debilidad mayor	Respuesta deficiente
2	Debilidad menor	Respuesta media
3	Fortaleza menor	Respuesta superior a la media
4	Fortaleza mayor	Respuesta excelente

El valor ponderado de cada factor se obtuvo multiplicando su peso por la calificación correspondiente.

La suma de los valores ponderados permite obtener un resultado entre 1,00 y 4,00, considerando 2,50 como punto de referencia para interpretar la situación general.

4.3 Matriz de Evaluación de Factores Internos (EFI)

La matriz EFI permite evaluar las fortalezas y debilidades asociadas a los recursos, capacidades técnicas y organización de Empre S.A.S.

Factor	Peso	Calificación	Ponderado
F1. Capacidad de desarrollar soluciones adaptadas a las necesidades del cliente	0,15	4	0,6
F2. Utilización de tecnologías	0,1	3	0,3

ampliamente documentadas			
F3. Posibilidad de ofrecer soluciones modulares y escalables	0,12	4	0,48
F4. Conocimientos en desarrollo web, bases de datos, infraestructura e interfaces	0,13	3	0,39
D1. Recursos humanos y económicos limitados	0,13	1	0,13
D2. Capacidad limitada para desarrollar varios proyectos simultáneamente	0,12	2	0,24
D3. Dependencia inicial de pocos clientes	0,13	1	0,13
D4. Falta de experiencia comercial y trayectoria	0,12	2	0,24
Total EFI	**1,00**		**2,51**

El resultado obtenido fue de 2,51 puntos, ubicándose ligeramente por encima del valor de referencia.

Las fortalezas alcanzaron un puntaje de 1,77, mientras que las debilidades sumaron 0,74.

Dentro de las principales fortalezas se destaca la capacidad de desarrollar soluciones personalizadas y modulares. En el caso de S.I.G.S.M., esto se refleja en la separación de sus funcionalidades en diferentes módulos y servicios especializados.

Por otra parte, la principal limitación identificada corresponde a los recursos humanos disponibles, debido al tamaño reducido del equipo. Esta situación exige una adecuada distribución de tareas y planificación de las actividades de desarrollo.

4.4 Matriz de Evaluación de Factores Externos (EFE)

La matriz EFE permite analizar las oportunidades y amenazas del entorno que pueden influir sobre el proyecto y su evolución.

Factor	Peso	Calificación	Ponderado
O1. Creciente necesidad de digitalización de procesos	0,15	4	0,6
O2. Mayor demanda de sistemas web para centralizar información	0,14	4	0,56
O3. Posibilidad de ampliar los servicios hacia otras instituciones	0,12	3	0,36
O4. Avances tecnológicos que permiten mejorar los servicios	0,09	3	0,27

A1. Competencia de empresas con mayor experiencia y recursos	0,14	2	0,28
A2. Cambios tecnológicos que requieren actualización constante	0,1	3	0,3
A3. Incremento de costos de infraestructura y equipamiento	0,1	2	0,2
A4. Riesgos de seguridad informática y protección de datos	0,16	3	0,48
Total EFI	**1,00**		**3,05**

El resultado obtenido fue de 3,05 puntos, indicando una posición favorable frente a las condiciones del entorno.

Las oportunidades alcanzaron un puntaje de 1,79, mientras que las amenazas sumaron 1,26.

Las principales oportunidades identificadas se relacionan con la necesidad de digitalizar procesos y centralizar información. Estas coinciden directamente con los objetivos de S.I.G.S.M., cuya finalidad es mejorar la gestión de documentación y traslados dentro del Hospital de Clínicas.

Entre las amenazas, adquiere especial importancia la seguridad informática y la protección de datos, debido al contexto institucional donde se utilizará la aplicación.

Por este motivo, durante el desarrollo se incorporaron mecanismos de autenticación, control de permisos, validación de datos y políticas de respaldo, destinados a reducir los riesgos asociados al tratamiento de la información.

4.5 Interpretación de resultados

Los resultados obtenidos permiten identificar una situación generalmente favorable para Empre S.A.S. y el desarrollo del proyecto.

Factor	Puntaje ponderado
Fortalezas	1,77
Debilidades	0,74
Oportunidades	1,79
Amenazas	1,26
Total EFI	2,51
Total EFE	3,05

Las fortalezas superan a las debilidades por 1,03 puntos, mientras que las oportunidades superan a las amenazas por 0,53 puntos.

Estos resultados permiten orientar la estrategia hacia el aprovechamiento de las capacidades técnicas del equipo y las oportunidades de digitalización existentes.

Sin embargo, el resultado de la matriz EFI se encuentra muy próximo al punto de referencia de 2,50, por lo que resulta importante continuar fortaleciendo la organización interna y la disponibilidad de recursos.

4.6 Estrategias derivadas del análisis

A partir de los factores identificados se plantean cuatro líneas estratégicas:

- **Estrategia FO (Fortalezas y Oportunidades)**: aprovechar la capacidad de desarrollar soluciones personalizadas y modulares para responder a las necesidades de digitalización de las instituciones. La arquitectura de S.I.G.S.M. permite contemplar futuras ampliaciones sin modificar completamente su estructura.
- **Estrategia DO (Debilidades y Oportunidades)**: utilizar la experiencia adquirida durante el desarrollo del proyecto para mejorar las capacidades del equipo, ampliar sus conocimientos y generar oportunidades para futuros desarrollos.
- **Estrategia FA (Fortalezas y Amenazas)**: aprovechar los conocimientos técnicos y las tecnologías utilizadas para implementar buenas prácticas de seguridad, mantenimiento y respaldo, reduciendo los riesgos asociados al tratamiento de información institucional.
- **Estrategia DA (Debilidades y Amenazas)**: mantener una planificación acorde con los recursos disponibles, priorizar las funcionalidades esenciales y utilizar herramientas de código abierto para evitar costos innecesarios.

4.7 Conclusiones del análisis FODA ponderado

El análisis FODA ponderado permitió identificar los principales factores que influyen en la capacidad de Empre S.A.S. para desarrollar y mantener S.I.G.S.M.

Los resultados obtenidos, con una matriz EFI de 2,51 y una matriz EFE de 3,05, muestran una situación favorable, aunque condicionada por las limitaciones propias de un equipo de desarrollo reducido.

Entre las principales fortalezas se destacan la capacidad de adaptación a los requerimientos, la utilización de tecnologías conocidas y la adopción de una arquitectura modular.

Asimismo, se identificaron oportunidades relacionadas con la digitalización de procesos y la mejora de los sistemas de gestión institucional.

Por otra parte, el análisis permitió reconocer la importancia de mantener una adecuada planificación, fortalecer los conocimientos técnicos y priorizar la seguridad de la información.

En conclusión, el análisis realizado respalda las decisiones adoptadas durante el desarrollo de S.I.G.S.M. y permite establecer estrategias para su mantenimiento y futuras ampliaciones.

5. Retrospectiva final del proyecto

5.1 Introducción y objetivos

La retrospectiva final del proyecto tiene como objetivo analizar el proceso de desarrollo de S.I.G.S.M., identificando los aspectos positivos, las dificultades encontradas y los aprendizajes adquiridos durante las distintas etapas de trabajo.

A diferencia de los apartados anteriores, centrados principalmente en los resultados técnicos, esta sección busca evaluar la experiencia del equipo desde una perspectiva organizativa, personal y profesional.

Para ello, se realizará una reflexión grupal sobre la planificación, la distribución de responsabilidades, la resolución de problemas y la evolución del proyecto. Posteriormente, cada integrante realizará una valoración individual de su

participación, las competencias adquiridas y los aspectos que considera necesario mejorar.

Este análisis permitirá reconocer las decisiones que contribuyeron positivamente al desarrollo del sistema y aquellos aspectos que podrían abordarse de manera diferente en futuros proyectos.

5.2 Retrospectiva grupal

5.2.1 Organización y distribución de tareas

El desarrollo de S.I.G.S.M. presentó diferentes etapas en cuanto a la organización y distribución de responsabilidades.

Inicialmente, el proyecto comenzó con un único integrante, Martín Cabrera, quien se encargó de establecer las primeras bases del sistema, seleccionar las tecnologías y comenzar la planificación y el desarrollo de la aplicación.

Posteriormente, se incorporó Ángel Sena, conformando el equipo definitivo de dos integrantes.

Esta incorporación requirió reorganizar las tareas y establecer una etapa inicial de adaptación, debido a que parte de las tecnologías seleccionadas, especialmente React, no eran conocidas previamente por el nuevo integrante.

Por este motivo, se decidió priorizar inicialmente su familiarización con el entorno de desarrollo, la estructura del proyecto y las herramientas utilizadas, mientras colaboraba principalmente con la documentación y otras actividades de apoyo.

Durante este proceso también se realizaron actividades conjuntas, como la configuración y reproducción del entorno de desarrollo, la preparación de la máquina virtual, la instalación de las herramientas necesarias y la comprobación del funcionamiento de la base de datos y la autenticación.

A medida que avanzó el proyecto, se mantuvo una distribución flexible de responsabilidades, adaptando las tareas a los conocimientos disponibles, las necesidades del desarrollo y los plazos de entrega.

La comunicación entre los integrantes permitió compartir avances, resolver dudas y coordinar las actividades pendientes.

Sin embargo, el tamaño reducido del equipo implicó que algunas responsabilidades técnicas se concentraran principalmente en uno de los integrantes, especialmente durante las etapas iniciales.

Esta experiencia permitió comprender la importancia de planificar la incorporación de nuevos participantes y establecer mecanismos que faciliten la transferencia de conocimientos dentro de un equipo de desarrollo.

5.2.2 Dificultades encontradas y soluciones aplicadas

Durante el desarrollo de S.I.G.S.M. surgieron diferentes dificultades, tanto técnicas como organizativas, que requirieron adaptar las decisiones inicialmente planteadas.

Una de las primeras dificultades estuvo relacionada con la configuración del entorno de desarrollo. La utilización de una máquina virtual con Fedora Server, junto con XAMPP, MariaDB y los servicios necesarios para ejecutar la API, exigió configurar correctamente la red, las conexiones y los distintos componentes del sistema.

Para resolver estas situaciones se realizaron pruebas de conectividad, revisiones de configuración y procedimientos documentados que posteriormente permitieron reproducir el entorno de desarrollo.

Otra dificultad importante estuvo relacionada con la integración del frontend y el backend. Fue necesario establecer mecanismos de autenticación, gestionar sesiones, validar información y coordinar las solicitudes HTTP entre React y PHP.

Durante este proceso se identificaron errores y comportamientos inesperados que fueron corregidos mediante pruebas progresivas y revisiones del código.

El módulo de traslados representó uno de los mayores desafíos técnicos debido a la cantidad de reglas de negocio involucradas. La asignación de vehículos y funcionarios, la verificación de disponibilidad, las restricciones de compatibilidad y las transiciones entre estados requirieron diseñar mecanismos que evitaran operaciones incorrectas o inconsistencias en la información.

Para facilitar su implementación y mantenimiento, se decidió separar estas responsabilidades en diferentes clases de servicio, evitando concentrar toda la lógica dentro de un único archivo.

También se presentaron limitaciones externas al equipo, especialmente la falta de acceso y documentación sobre determinados servicios del Hospital de Clínicas. Esta situación impidió completar algunas integraciones previstas inicialmente, como la consulta de información de pacientes mediante sistemas institucionales.

Ante esta limitación, se priorizó el desarrollo de las funcionalidades que podían implementarse y verificarse de manera independiente, dejando documentadas aquellas integraciones pendientes.

Finalmente, la disponibilidad de tiempo y la necesidad de compatibilizar el desarrollo con las demás actividades académicas exigieron priorizar las funcionalidades esenciales y ajustar la planificación a medida que avanzaba el proyecto.

En conjunto, estas dificultades permitieron adquirir experiencia en la resolución de problemas reales y comprender la importancia de mantener una arquitectura organizada y una planificación flexible.

5.2.3 Aspectos positivos y aprendizajes del equipo

Uno de los principales aspectos positivos del proyecto fue la capacidad de transformar los requerimientos iniciales en una aplicación funcional, integrando distintas tecnologías y conocimientos adquiridos durante la formación.

El desarrollo permitió aplicar conceptos de programación frontend y backend, diseño de bases de datos, administración de servidores, seguridad informática, control de versiones e ingeniería de software.

Asimismo, se considera positiva la adopción de una arquitectura modular, ya que facilitó la incorporación progresiva de nuevas funcionalidades y permitió mantener una organización más clara del código conforme aumentaba la complejidad del sistema.

Otro aspecto destacable fue la utilización de GitHub para centralizar el código fuente y mantener un historial de los cambios realizados, facilitando el trabajo colaborativo y el seguimiento del desarrollo.

La documentación técnica también constituyó un aprendizaje importante. A lo largo del proyecto se elaboraron diagramas, descripciones de arquitectura,

procedimientos de instalación, políticas de respaldo y registros de pruebas, comprendiendo que el desarrollo de software no se limita únicamente a programar.

En cuanto al trabajo en equipo, la incorporación de un segundo integrante permitió desarrollar habilidades relacionadas con la comunicación técnica, la explicación de conceptos y la coordinación de responsabilidades.

Finalmente, las demostraciones realizadas y los comentarios recibidos durante las instancias de evaluación permitieron obtener una perspectiva externa sobre el funcionamiento de la aplicación e identificar posibles mejoras.

5.2.4 Aspectos a mejorar

A pesar de los resultados obtenidos, durante la retrospectiva se identificaron diferentes aspectos que podrían mejorarse en futuros proyectos.

En primer lugar, sería conveniente establecer una distribución de tareas más equilibrada desde las etapas iniciales, evitando que una parte importante del desarrollo dependa de un único integrante.

También resultaría beneficioso reservar más tiempo para la incorporación y capacitación de nuevos participantes, de manera que puedan familiarizarse con las tecnologías y contribuir de forma progresiva al desarrollo.

Otro aspecto a mejorar corresponde a la planificación de las pruebas. Si bien se realizaron comprobaciones durante la implementación, habría sido conveniente definir desde el comienzo un plan de testing más estructurado, manteniendo un registro continuo de los resultados.

Asimismo, se considera importante actualizar la documentación de manera simultánea con el desarrollo, evitando acumular modificaciones pendientes para las fechas próximas a las entregas.

Por último, sería recomendable establecer una planificación más detallada de las funcionalidades, diferenciando desde el inicio aquellas que dependen exclusivamente del equipo de las que requieren información, acceso o colaboración de terceros.

Estas mejoras permitirían reducir dificultades organizativas, facilitar el seguimiento del proyecto y aprovechar de manera más eficiente el tiempo disponible.

5.3 Retrospectivas individuales

5.3.1 Martín Cabrera

Mi participación en S.I.G.S.M. comenzó desde las primeras etapas del proyecto, asumiendo responsabilidades relacionadas con la planificación inicial, la selección de tecnologías, la organización del repositorio y el desarrollo de la aplicación.

Uno de los principales desafíos personales fue tener que abordar distintas áreas del desarrollo de software, incluyendo frontend, backend, bases de datos e infraestructura, procurando que todas funcionaran de manera integrada.

A medida que avanzó el proyecto, pude profundizar mis conocimientos en React, PHP, MariaDB y administración de servidores Linux, además de adquirir mayor experiencia en el diseño de APIs y la organización de código mediante servicios especializados.

Considero que uno de los aprendizajes más importantes fue comprender la necesidad de diseñar una arquitectura que facilite el mantenimiento y la ampliación del sistema. Durante el desarrollo surgieron situaciones en las que fue necesario reorganizar funcionalidades previamente implementadas, demostrando que una solución que funciona correctamente no siempre es la más adecuada desde el punto de vista de su estructura.

La incorporación de un segundo integrante también representó una experiencia importante, ya que implicó explicar las decisiones técnicas tomadas, compartir conocimientos y reorganizar las actividades para facilitar el trabajo conjunto.

En cuanto a los aspectos a mejorar, considero que debería haber dedicado más tiempo a la planificación y distribución de responsabilidades, así como a establecer procedimientos de testing y documentación desde las primeras etapas.

En ocasiones, la prioridad estuvo centrada en conseguir que determinadas funcionalidades funcionaran correctamente, dejando para momentos posteriores actividades relacionadas con la documentación y las pruebas formales.

En conclusión, el proyecto representó una oportunidad para aplicar conocimientos adquiridos durante la formación y enfrentar problemas propios de un desarrollo de software real. Más allá del resultado final, considero especialmente valioso haber adquirido una mayor capacidad para analizar problemas, tomar decisiones técnicas y desarrollar soluciones de manera autónoma.

5.3.2 Ángel Sena

Mi incorporación a S.I.G.S.M. se produjo cuando el proyecto ya contaba con sus primeras bases definidas, lo que implicó integrarme a un trabajo en marcha y adaptarme a decisiones técnicas que ya habían sido tomadas.

Uno de los principales desafíos personales fue familiarizarme con tecnologías que no conocía previamente, especialmente React, así como comprender la estructura del repositorio, la organización del backend y el funcionamiento del entorno de desarrollo. Para ello, dediqué la etapa inicial a estudiar el proyecto existente y a reproducir el entorno junto a mi compañero, participando en la preparación de la máquina virtual, la instalación de las herramientas necesarias y la verificación del funcionamiento de la base de datos y la autenticación.

Durante gran parte del proyecto, mi aporte se centró en la documentación y en actividades de apoyo al desarrollo. Esta tarea me permitió comprender el trabajo de mi compañero más a fondo. Elaborar diagramas, describir la arquitectura, registrar procedimientos y analizar métricas me obligó a entender en profundidad el funcionamiento del sistema y las decisiones tomadas en cada etapa.

Considero que uno de mis aprendizajes más importantes fue aprender a incorporarme a un proyecto existente: leer y comprender código escrito por otra persona, realizar las consultas adecuadas y adaptarme a una forma de trabajo ya establecida. Esta situación es habitual en el ámbito profesional y me permitió desarrollar constancia, paciencia y capacidad de autoaprendizaje.

También valoro la experiencia de trabajar en un equipo reducido, donde la comunicación resultó bastante fácil para coordinar las tareas, compartir avances y resolver dudas en conjunto.

En cuanto a los aspectos a mejorar, reconozco que mi participación en el desarrollo técnico fue menor a la que me hubiera gustado, principalmente por el tiempo que requirió el aprendizaje de las tecnologías utilizadas. En futuros proyectos me propongo involucrarme antes en tareas de programación, aunque sean de menor

alcance, para ganar experiencia práctica de forma progresiva y contribuir a una distribución de responsabilidades más equilibrada.

En conclusión, el proyecto representó para mí una experiencia de crecimiento tanto técnico como personal. Más allá de las dificultades iniciales, considero especialmente valioso el esfuerzo realizado para ponerme al día con el proyecto, así como los conocimientos adquiridos en desarrollo web, documentación técnica y trabajo en equipo, que podré aplicar en futuros proyectos de desarrollo de software.

5.4 Conclusiones de la retrospectiva

La retrospectiva final permitió analizar el proceso de desarrollo de S.I.G.S.M. desde una perspectiva técnica, organizativa y personal.

A lo largo del proyecto se presentaron dificultades relacionadas con el aprendizaje de tecnologías, la configuración del entorno, la integración de componentes, la implementación de reglas de negocio y la distribución del tiempo disponible.

Sin embargo, estas situaciones permitieron desarrollar habilidades de análisis, resolución de problemas, comunicación y adaptación a cambios.

Entre los principales logros se destacan la implementación de las funcionalidades del sistema, la adopción de una arquitectura modular y la adquisición de experiencia en el desarrollo de una aplicación web que integra múltiples tecnologías.

Asimismo, la experiencia permitió comprender la importancia de distribuir adecuadamente las responsabilidades, documentar las decisiones técnicas y mantener una planificación que contemple tanto el desarrollo como las pruebas y el mantenimiento.

En conclusión, S.I.G.S.M. constituyó una experiencia formativa que permitió aplicar conocimientos teóricos en un contexto práctico y adquirir nuevas competencias técnicas y profesionales, aportando aprendizajes que podrán ser utilizados en futuros proyectos de desarrollo de software.

6. Evaluación de la experiencia del cliente

6.1 Introducción y objetivos

La evaluación de la experiencia del cliente tiene como finalidad analizar la recepción de S.I.G.S.M. durante las instancias de presentación y demostración del sistema, considerando los comentarios, observaciones y sugerencias recibidas.

Este proceso permite obtener una valoración externa del producto desarrollado, identificar posibles oportunidades de mejora y comprobar si las funcionalidades implementadas responden adecuadamente a las necesidades planteadas inicialmente.

En el contexto del proyecto académico, se realizaron demostraciones de la aplicación ante los docentes responsables de su seguimiento y evaluación, quienes proporcionaron observaciones sobre su funcionamiento y posibles mejoras.

Cabe destacar que esta evaluación corresponde a la retroalimentación obtenida durante las instancias académicas y no constituye una validación formal del sistema por parte del Hospital de Clínicas, institución destinataria del proyecto.

Los principales objetivos de esta evaluación son identificar la percepción general de la aplicación, analizar los comentarios recibidos y establecer posibles mejoras para futuras versiones.

6.2 Presentación y demostración del sistema

Durante las instancias de seguimiento del proyecto se presentó el estado de desarrollo de S.I.G.S.M., mostrando las principales funcionalidades implementadas y su funcionamiento desde la interfaz web.

La demostración permitió visualizar la integración entre el frontend, el backend y la base de datos, así como las operaciones disponibles en los diferentes módulos.

Entre las funcionalidades desarrolladas se encuentran:

- **Autenticación y administración de funcionarios**: inicio de sesión, gestión de cuentas, asignación de roles y administración de solicitudes de acceso.
- **Gestión documental**: administración de categorías, carga y modificación de documentos, visualización de archivos y acceso a la información publicada.
- **Gestión de encuestas**: creación y administración de encuestas, configuración de preguntas, registro de respuestas y consulta de resultados.
- **Gestión de traslados**: creación de solicitudes, asignación de vehículos y funcionarios, consulta de disponibilidad de recursos, seguimiento de estados y visualización del historial.
- **Administración de vehículos**: registro, consulta y actualización de los vehículos disponibles para los traslados.

A través de estas funcionalidades fue posible presentar una visión general del sistema y demostrar cómo las operaciones desarrolladas contribuyen a la digitalización y organización de determinados procesos institucionales.

La demostración también permitió recibir comentarios sobre situaciones de uso que podrían requerir validaciones adicionales o modificaciones en el comportamiento de la aplicación.

6.3 Valoración y comentarios recibidos

La recepción general de S.I.G.S.M. durante las demostraciones realizadas fue positiva.

Los docentes manifestaron una valoración favorable del trabajo presentado, mostrando interés por las funcionalidades desarrolladas y por el grado de avance alcanzado por el proyecto.

Esta recepción resultó especialmente significativa considerando que el sistema evolucionó desde los prototipos y requerimientos iniciales hasta una aplicación funcional, con distintos módulos integrados y operaciones conectadas a una base de datos.

Asimismo, durante la presentación surgieron sugerencias destinadas a mejorar determinados aspectos del funcionamiento, principalmente relacionados con las reglas de negocio y las validaciones de los formularios.

Estas observaciones permitieron analizar el sistema desde una perspectiva diferente a la del equipo de desarrollo, considerando situaciones que pueden presentarse durante su utilización.

No se realizó una encuesta formal de satisfacción ni se estableció una calificación cuantitativa de la experiencia. Por lo tanto, la valoración presentada corresponde a una evaluación cualitativa basada en los comentarios recibidos durante las demostraciones.

6.4 Sugerencias de mejora

Durante las instancias de evaluación se identificaron oportunidades de mejora que podrían incorporarse en futuras versiones del sistema.

Una de las sugerencias recibidas estuvo relacionada con el registro de nuevas solicitudes de traslado, especialmente cuando estas tienen carácter urgente.

Se planteó la necesidad de que, al registrar un traslado con prioridad urgente, el sistema solicite obligatoriamente una fecha y hora para su ejecución.

Esta observación responde a la importancia de contar con información temporal precisa que permita organizar las solicitudes y facilitar la coordinación de los recursos disponibles.

La propuesta consiste en incorporar una validación condicional al formulario de creación de traslados:

- Cuando la solicitud tenga prioridad urgente, la fecha y hora requerida deberán ser campos obligatorios.
- El sistema deberá impedir el registro de una solicitud urgente si no se proporciona esta información.
- Se deberá mostrar un mensaje de validación que informe claramente al funcionario sobre los datos faltantes.
- La validación deberá aplicarse tanto en el frontend como en el backend, garantizando que la restricción no pueda evitarse mediante solicitudes directas a la API.

Esta mejora permitiría reforzar la consistencia de las solicitudes y proporcionar información más precisa para la posterior asignación de vehículos y funcionarios.

La sugerencia se considera pertinente y técnicamente viable dentro de la arquitectura actual, ya que el módulo de traslados dispone de mecanismos de validación de datos y de servicios especializados para procesar las solicitudes.

Su incorporación queda identificada como una mejora propuesta a partir de la retroalimentación recibida.

6.5 Análisis de la retroalimentación

Los comentarios recibidos durante las demostraciones permitieron obtener una perspectiva externa sobre el funcionamiento de S.I.G.S.M. y reconocer la importancia de evaluar la aplicación más allá de su implementación técnica.

La valoración positiva de los docentes representa un resultado favorable para el equipo, ya que permite reconocer el avance conseguido y la capacidad de integrar las funcionalidades desarrolladas en un sistema coherente.

Sin embargo, las sugerencias recibidas también evidencian que una aplicación puede funcionar correctamente desde el punto de vista técnico y, aun así, presentar oportunidades de mejora relacionadas con los procesos reales de sus usuarios.

El ejemplo de los traslados urgentes demuestra la importancia de considerar las reglas de negocio y las necesidades operativas al diseñar formularios y validaciones.

Si bien el sistema permite registrar solicitudes de traslado con diferentes prioridades, exigir información temporal en determinadas situaciones contribuiría a mejorar la calidad de los datos registrados y facilitaría la planificación de los recursos.

Esta observación también pone de manifiesto la utilidad de realizar demostraciones durante el desarrollo, ya que permiten identificar necesidades que no siempre quedan completamente definidas en las primeras etapas de relevamiento.

Por otra parte, la ausencia de una evaluación formal por parte de los usuarios finales del Hospital de Clínicas constituye una limitación que deberá considerarse antes de implementar el sistema en un entorno institucional real.

En ese contexto, sería recomendable realizar pruebas de aceptación con funcionarios que participen directamente en los procesos de documentación y gestión de traslados, permitiendo comprobar que el funcionamiento de la aplicación se adapte a sus procedimientos habituales.

6.6 Conclusiones de la evaluación

La evaluación de la experiencia permitió obtener una valoración externa favorable sobre el desarrollo de S.I.G.S.M., destacándose la recepción positiva de los docentes durante las demostraciones realizadas.

Las observaciones recibidas contribuyeron a identificar oportunidades de mejora, especialmente en el módulo de traslados, donde se propuso incorporar una validación obligatoria de fecha y hora para las solicitudes urgentes.

Esta sugerencia demuestra la importancia de complementar las pruebas funcionales con instancias de evaluación que permitan analizar la aplicación desde la perspectiva de quienes podrían utilizarla.

Asimismo, la experiencia permitió reconocer el valor de la retroalimentación durante el desarrollo de software, ya que facilita detectar necesidades adicionales y realizar ajustes orientados a mejorar la utilidad del producto.

En conclusión, la recepción obtenida durante las demostraciones representa un resultado positivo para el proyecto y proporciona información útil para continuar perfeccionando S.I.G.S.M.

No obstante, la validación definitiva de su adecuación a los procesos institucionales deberá realizarse con representantes y usuarios del Hospital de Clínicas antes de su utilización en un entorno de producción.

7. Conclusiones generales del proyecto

7.1 Cumplimiento de los objetivos

El desarrollo de S.I.G.S.M. tuvo como objetivo principal proporcionar una solución informática que permitiera mejorar determinados procesos administrativos y operativos del Hospital de Clínicas, especialmente aquellos relacionados con la gestión documental y la organización de traslados institucionales.

A lo largo del proyecto se avanzó progresivamente desde el relevamiento de requerimientos y la elaboración de prototipos hasta el desarrollo de una aplicación web funcional, integrando diferentes módulos y tecnologías.

Entre los objetivos alcanzados se destacan la implementación de una arquitectura cliente-servidor, el desarrollo de una API REST, la utilización de una base de datos relacional y la construcción de una interfaz web que permite interactuar con las funcionalidades del sistema.

Asimismo, se incorporaron mecanismos de autenticación, administración de funcionarios y control de permisos, permitiendo organizar el acceso a las diferentes operaciones según los roles asignados.

Desde el punto de vista de la Ingeniería de Software, también se cumplieron objetivos relacionados con el análisis de requerimientos, la elaboración y actualización de diagramas, la aplicación de métricas, la documentación técnica y la ejecución de pruebas funcionales.

En términos generales, se considera que los objetivos principales del proyecto fueron alcanzados dentro del entorno de desarrollo y pruebas, aunque determinadas integraciones y validaciones institucionales permanecen pendientes.

7.2 Resultados y funcionalidades desarrolladas

Como resultado del trabajo realizado, S.I.G.S.M. cuenta con diferentes módulos que permiten administrar información y ejecutar operaciones relacionadas con los procesos contemplados en el proyecto.

El módulo de documentación permite organizar categorías, cargar y administrar documentos PDF, modificar su información y facilitar el acceso a los archivos publicados.

Asimismo, se incorporó la gestión de encuestas, permitiendo crear formularios, administrar preguntas y opciones, registrar respuestas y consultar los resultados obtenidos.

Por otra parte, el módulo de traslados permite registrar solicitudes, consultar los recursos disponibles, asignar vehículos y funcionarios, realizar el seguimiento de los estados y consultar el historial de traslados completados o cancelados.

También se desarrollaron funcionalidades para la administración de vehículos, funcionarios, roles y solicitudes de acceso.

Desde el punto de vista técnico, uno de los principales resultados fue la adopción de una arquitectura modular, separando las responsabilidades del backend mediante endpoints, handlers y clases de servicio especializadas.

Esta organización permitió mantener una estructura más clara a medida que aumentaba la cantidad de funcionalidades, facilitando la localización de errores, el mantenimiento y la incorporación de nuevas operaciones.

Adicionalmente, se configuró un entorno de desarrollo basado en Fedora Server, PHP, MariaDB y React, y se incorporaron herramientas para facilitar la instalación y el despliegue del sistema.

En cuanto a la validación del software, se ejecutaron 36 casos de prueba funcionales y exploratorios, obteniéndose resultados satisfactorios en todos los escenarios evaluados.

Estos resultados aportan evidencia favorable sobre el funcionamiento de las funcionalidades desarrolladas, aunque no sustituyen las pruebas de aceptación que deberán realizarse en un entorno institucional.

7.3 Limitaciones y funcionalidades pendientes

A pesar de los avances alcanzados, durante el desarrollo se presentaron determinadas limitaciones que condicionaron el alcance final del proyecto.

Una de las principales dificultades estuvo relacionada con la falta de acceso a los sistemas internos del Hospital de Clínicas. Inicialmente se contemplaba la posibilidad de consultar información de pacientes mediante servicios institucionales,

pero no se proporcionaron los accesos ni la documentación técnica necesarios para realizar dicha integración.

Por este motivo, se decidió priorizar las funcionalidades que podían desarrollarse y comprobarse de manera independiente, dejando esta integración como una tarea pendiente.

Otra limitación estuvo relacionada con la disponibilidad de recursos humanos y tiempo. El proyecto fue desarrollado por un equipo reducido, cuya composición definitiva se estableció después de iniciadas las actividades, lo que requirió reorganizar responsabilidades y dedicar tiempo al aprendizaje de las tecnologías utilizadas.

Asimismo, aunque la aplicación fue evaluada durante las instancias académicas y recibió comentarios favorables, todavía no se realizó una validación formal con los usuarios finales del Hospital de Clínicas.

Por lo tanto, no es posible garantizar que todos los procedimientos implementados se adapten completamente a las necesidades operativas de la institución sin realizar previamente pruebas de aceptación con sus funcionarios.

Finalmente, el despliegue definitivo sobre la infraestructura del hospital constituye una etapa diferente del desarrollo realizado en el entorno de pruebas y requiere verificar las condiciones de instalación, conectividad, seguridad y mantenimiento correspondientes.

Estas limitaciones no impiden valorar los resultados obtenidos, pero deben considerarse antes de una posible puesta en producción.

7.4 Posibles mejoras y ampliaciones futuras

A partir de la experiencia adquirida durante el desarrollo y de los comentarios recibidos en las demostraciones, se identificaron diferentes oportunidades de mejora para futuras versiones de S.I.G.S.M.

Entre las principales se encuentran:

- **Integración con sistemas institucionales**: incorporar la consulta de información de pacientes mediante los servicios proporcionados por el Hospital de Clínicas, una vez que se disponga de los accesos y la documentación necesarios.
- **Mejoras en la gestión de traslados urgentes**: establecer como obligatoria la fecha y hora requerida cuando se registre una solicitud con prioridad urgente, aplicando las validaciones correspondientes tanto en el frontend como en el backend.
- **Ampliación de las pruebas**: incorporar nuevos casos de prueba, ampliar la automatización del testing y realizar pruebas de aceptación con funcionarios de la institución.
- **Fortalecimiento de la seguridad**: continuar revisando los mecanismos de autenticación, autorización, protección de datos y configuración del entorno antes de su utilización en producción.
- **Mejoras en la experiencia de usuario**: incorporar los ajustes que surjan de las pruebas con usuarios reales, especialmente en los formularios y procedimientos utilizados con mayor frecuencia.
- **Mantenimiento y escalabilidad**: continuar aprovechando la estructura modular del sistema para facilitar futuras modificaciones e incorporar nuevas funcionalidades sin afectar innecesariamente los componentes existentes.

Estas propuestas permitirían avanzar hacia una versión más completa y adaptada a las necesidades de la institución, aprovechando la base técnica desarrollada durante el proyecto.

7.5 Reflexión y cierre del proyecto

El desarrollo de S.I.G.S.M. representó una experiencia de aprendizaje que permitió aplicar de manera práctica conocimientos adquiridos durante la formación, integrando diferentes áreas de las Tecnologías de la Información.

A lo largo del proyecto fue necesario abordar actividades relacionadas con el análisis de requerimientos, diseño de bases de datos, programación frontend y backend, administración de servidores, control de versiones, seguridad informática, documentación y validación del software.

Esta experiencia permitió comprender que el desarrollo de una aplicación no consiste únicamente en implementar funcionalidades, sino también en analizar necesidades, tomar decisiones técnicas, organizar responsabilidades, resolver problemas y comprobar que los resultados obtenidos cumplan con los objetivos establecidos.

Asimismo, las dificultades surgidas durante el proceso demostraron la importancia de mantener una planificación flexible, documentar las decisiones tomadas y adaptar las soluciones cuando aparecen nuevas necesidades o limitaciones.

La revisión de las métricas del proyecto permitió observar su evolución entre las diferentes entregas, mientras que las pruebas funcionales y las demostraciones realizadas proporcionaron información útil sobre el funcionamiento de la aplicación y sus posibilidades de mejora.

Desde el punto de vista del trabajo en equipo, el proyecto también permitió desarrollar habilidades de comunicación, coordinación, transferencia de conocimientos y resolución conjunta de problemas.

En conclusión, se considera que S.I.G.S.M. alcanzó los objetivos principales establecidos para esta etapa, proporcionando una aplicación funcional y una base técnica organizada para su posible evolución.

Si bien todavía existen aspectos pendientes antes de una implementación definitiva en el Hospital de Clínicas, el trabajo realizado constituye un resultado satisfactorio tanto desde el punto de vista técnico como académico.

Finalmente, el proyecto permitió consolidar conocimientos, adquirir experiencia práctica y comprender con mayor profundidad las responsabilidades que implica participar en el desarrollo de un sistema informático destinado a resolver necesidades reales.